

Agile과 MSA의 이해

울산_4반 이름: 박인우

Chapter 1. 개인 요약 정리

전통적 개발은 현재 요구사항을 바탕으로 미래의 완제품을 최대한 예측하고 전체 계획·설계를 먼저 확정된 뒤 개발하는 방식. 반면 애자일은 불확실성과 변경을 전제로, 현재 만들 수 있는 작은 가치 있는 결과물을 구현하고 피드백을 받아 다음 결과물을 결정. 따라서 요구사항이 자주 변하거나 사용자 반응을 사전에 예측하기 어려운 AI 서비스에 특히 적합함. 예를 들어 게임 뉴스 서비스라면 처음부터 개인화·추천까지 완성하기보다 뉴스 수집 → AI 요약 → Feed를 먼저 만든 뒤 실제 반응을 보고 Topic 통합, 중요도, 개인화 등을 점진적으로 확장한다.

Scrum은 애자일을 실제 팀에서 운영하는 대표적인 프레임워크로, Product Backlog → Sprint Planning → Sprint → Review → Retrospective를 반복하며 매 Sprint마다 실제 동작하는 Increment를 만든다. 요구사항은 User Story로 관리하며 As a 사용자, I want 기능, So that 가치 형태로 표현한다. Epic을 User Story로 나누고 MoSCoW로 우선순위를 정한 뒤, INVEST 기준과 Acceptance Criteria를 통해 Sprint 직전에 필요한 수준까지만 상세화한다. 즉 백로그는 고정된 계획이 아니라 지속적으로 수집·재우선순위화되는 팀의 작업 기준이다.

MSA 실습도 이 관점으로 볼 수 있다. 처음부터 user, course, enrollment, payment, recommend, Kafka를 모두 조금씩 만드는 것이 아니라 Sprint 1에서는 3개 정도의 핵심 도메인을 골라 사용자가 처음부터 끝까지 사용할 수 있는 Walking Skeleton을 완성한다. 예를 들어 회원가입 → 강의 조회 → 수강신청 흐름을 먼저 만든다. 기능을 분석해 담당 서비스를 정하고, 외부나 다른 서비스가 기능을 사용해야 하면 GET /api/courses, POST /api/enrollments처럼 HTTP Method + 경로 형태의 Endpoint를 설계한다. 같은 Spring Boot 내부는 DI와 메서드 호출로 연결하지만, 서로 분리된 MSA 서비스·컨테이너 간에는 REST API를 사용하고, 결제 완료처럼 사건을 전달하는 흐름은 Kafka를 사용할 수 있다. 이후 Sprint 2에서 payment.completed, 결제 서비스, 추천 서비스를 확장한다.

따라서 “어떤 사용자에게 어떤 가치를 제공할 것인가 → 어떤 서비스가 담당할 것인가 → 어떤 Endpoint와 통신 방식이 필요한가 → 작은 Increment로 무엇부터 검증할 것인가”를 설계하는 사고방식이 중요하다. 특히 프로젝트를 기획할 때는 변경 가능성, 불확실성, 피드백 필요성을 보고 이 서비스에 애자일이 적합한가, 그리고 MSA로 분리했을 때 독립 배포·확장·변경이라는 실제 가치가 생기는지를 판단해야 한다.

예를 들어,

MSA가 적용 가능할 서비스는

대형 쇼핑몰(기능이 많고 도메인이 나뉨), 넷플릭스같은 스트리밍(기능마다 트래픽 차이가 큼), 배달/택시 플랫폼(실시간 처리가 많음), 금융/결제 서비스(장애를 격리해야함), ai 기능이 붙은 플랫폼(특정 기능만 자주 변경됨)

반면, 반도체 설계처럼 RTL, 합성, 배치·배선, 타이밍 검증 결과가 서로 강하게 영향을 주고 하나의 변경으로 여러 단계를 다시 검증해야 하는 영역은 기능 간 독립성이 낮다. 이런건 msa하면 안됨.

Chapter 2. 실습 서비스가 어떤 가치를 주나

1. 실습 개요

본 프로젝트는 Spring Boot, FastAPI, MariaDB, Kafka, Eureka, API Gateway를 활용한 MSA 기반 온라인 교육 플랫폼이다. 수강생은 회원가입·로그인·강의 검색·수강신청·결제·추천 기능을 사용할 수 있고, 강사는 강의를 등록할 수 있다. 시스템은 결제 완료 후 수강 상태를 자동으로 활성화하고, 수강 완료 정보를 바탕으로 추천을 갱신하도록 설계되어 있다.

단일 서버에서 모든 기능을 처리하지 않고 User, Course, Enrollment, Payment, Recommend로 서비스를 분리하여 각 도메인이 자신의 기능과 데이터를 담당하도록 구성

2. 고객에게 제공하는 가치

통합된 온라인 학습 흐름

수강생은 강의를 조회한 뒤 수강신청과 결제를 진행하고 이후 자신의 수강 상태까지 관리할 수 있다. 여러 기능이 하나의 사용자 흐름으로 연결되어 온라인 교육 서비스의 기본적인 학습 과정을 제공한다.

결제 이후 자동화된 수강 처리

Payment Service에서 결제가 완료되면 **payment.completed** 이벤트를 **Kafka**로 전달하고 **Enrollment Service**가 이를 받아 수강 상태를 **ACTIVE**로 변경한다. 사용자가 별도로 처리하지 않아도 서비스 간 업무가 자동으로 이어진다.

개인별 강의 추천

Recommend Service는 사용자의 수강 완료 강의 카테고리를 분석하고 같은 카테고리의 미수강 강의를 찾아 추천한다. 따라서 단순 강의 목록보다 사용자 이력을 반영한 탐색 경험을 제공할 수 있다.

서비스별 독립적인 확장 가능성

회원, 강의, 수강, 결제, 추천을 서로 다른 서비스로 나누었기 때문에 특정 기능의 변경이 다른 영역 전체에 미치는 영향을 줄이고 서비스 단위 확장을 고려할 수 있다.

3. 주요 적용 가능 사례

온라인 교육 업체: 강의 등록, 수강신청, 결제, 수강생 관리

기업 사내교육 플랫폼: 직원별 교육 신청 및 수강 이력 관리

소규모 강의 플랫폼: 강사별 강좌 판매와 결제 관리

에 적용할 수 있고 이 서비스를 응용하면,

중고 거래 서비스: **Course** → 상품, **Enrollment** → 거래/구매, **Payment** → 결제처럼 도메인만 바꾸면 회원·상품·거래·결제 서비스를 분리할 수 있음.

공연 티켓 판매 서비스: **Course** → 공연, **Enrollment** → 예매, **Payment** → 결제로 대응시킬 수 있고, 결제 완료 후 예매 확정 같은 이벤트 처리에도 **Kafka** 구조를 활용할 수 있음.

게임 스토어 서비스: **Course** → 게임 상품, **Enrollment** → 구매/거래, **Payment** → 결제, **Recommend** → 관심 게임·구매 이력 기반 상품 추천처럼 도메인을 변경하면 회원·상품·거래·결제·추천 서비스를 분리해 적용할 수 있음.

등 까지 확장할 수 있다

Chapter 3. 코드 이해

1. 기능을 보고 서비스 경계를 나눔

User Service → 회원가입, 로그인, 인증

Course Service → 강의 등록/조회/검색

Enrollment Service → 수강신청, 수강 상태

Payment Service → 결제

Recommend Service → 추천

2. 서비스 내부 구조는 우리가 전에 배운 **Spring**이랑 거의 같다

HTTP 요청 → CourseController → CourseService → CourseRepository → courses DB

기존 Spring Boot 애플리케이션 여러 개를 기능별로 분리한 것

3. 엔드포인트(Rest API)

외부에서 이 서비스의 특정 기능을 사용할 수 있도록 만들어 놓은 HTTP 접점

{누가? → 사용자}, {무엇을? → 강의 목록을 조회}, {어느 서비스가 담당? → Course Service}, {데이터를 읽는가? → YES}, {HTTP Method? → GET}, {Endpoint? → /api/courses}

GET /api/courses

{사용자가 수강신청을 한다}, {담당 → Enrollment Service}, {새로운 Enrollment 생성 → POST},
{Endpoint → /api/enrollments}

POST /api/enrollments

4. REST API는 프론트 → 백엔드만 쓰는 게 아니다

Enrollment가 수강신청을 처리하려면, "이 강의가 진짜 존재하는가?"를 Course Service에 물어봐야 함

그런데 둘은 별도의 프로그램이니까

EnrollmentService 객체 → CourseService 객체

이렇게 @Autowired로 직접 연결할 수 없음.

대신:

Enrollment Service → REST → Course Service 로 호출.

그리고 구현 구조도:

EnrollmentService

└─ CourseServiceClient

└─ PaymentServiceClient

처럼 다른 서비스를 호출하는 Client를 따로 둬

5. 엔드포인트(Rest API)

internal 엔드포인트

API 설계할 때: 사용자가 사용하는 API

GET /api/courses/3 와

서비스끼리 사용하는 API

GET /api/courses/internal/exists/3

를 구분할 수 도 있음

6. REST냐 Kafka냐도 "기능 요구사항"에서 결정

즉시 결과가 필요하면:

Enrollment

↓ REST

Course

"강의 있어?" ← "있음", REST.

Payment

↓

"결제 완료됨"

↓ Kafka

Enrollment

처럼 이미 발생한 사실을 다른 서비스에 알려주는 것은 **Kafka** 이벤트로 처리

ex) `payment.completed`를 Payment가 발행하고 Enrollment가 받아 수강 상태를 **ACTIVE**로
변경하도록 설계

7. Gateway는 엔드포인트 앞의 안내소

각 서비스가:

User :8081, Course :8082, Enrollment :8083, Payment :8084, Recommend :8085

에 따로 떠 있지만 클라이언트가 전부 기억하게 만들면 복잡

그래서 Gateway를 8080 하나로 두고 보내줌.

Client

↓

Gateway :8080

|

└─ /users/** → User

└─ /courses/** → Course

└─ /enrollments/** → Enrollment

└─ /payments/** → Payment

└─ /recommend/** → Recommend

Gateway를 단일 진입점, 라우팅, 인증 필터 역할로 정의

즉 사용자 입장에서는

GET localhost:8080/courses

만 보내더라도 내부에서는:

Gateway → Course Service :8082 로 라우팅

8. 흐름 정리

① 사용자 기능 찾기 "수강신청"

② 담당 서비스 결정 Enrollment Service

③ 데이터/처리 결정 Enrollment 생성

④ 외부에서 호출해야 하나? YES

⑤ Endpoint 설계 POST /api/enrollments

⑥ 다른 서비스 정보가 필요한가? 강의 존재 확인 필요

⑦ 서비스 간 통신 방식 결정 즉시 답 필요 → REST → GET /api/courses/internal/exists/{id}

⑧ 이후 사건을 알려야 하나? → Kafka payment.completed

Chapter 4. 에자일 방법론 연계

1. 먼저 전체 요구사항을 **Backlog**로 만든다

[Backlog] - 회원가입하고 싶다 - 로그인하고 싶다 - 강의 목록을 보고 싶다 - 강의 상세정보를 보고 싶다 - 수강신청하고 싶다 - 결제하고 싶다 - 내 수강목록을 보고 싶다 - 강의를 추천받고 싶다

2. Sprint 1 — 아주 작은 완성품 만들기

학생이 강의를 보고 수강신청까지 할 수 있게 하자.

그러면 필요한 서비스만 고르면:

User, Course, Enrollment

Payment, Recommend는 일단 뒤로 미룬다.

그리고 기능에서 필요한 엔드포인트를 역으로 뽑는다

회원가입 필요 → POST /api/users/register

강의목록 필요 → GET /api/courses

강의 상세 필요 → GET /api/courses/{id}

수강신청 필요 → POST /api/enrollments

엔드포인트를 사용자 기능에서 도출

사용자 요구 → 필요한 기능 → 담당 서비스 → Endpoint → Controller / Service / Repository 구현

그리고 수강신청을 만들어보다가 새로운 요구가 보임

Enrollment Service 입장에서:

"잠깐, 사용자가 보내준 **courseId**가 진짜 있는 강의인지 어떻게 확인하지?"

라는 문제가 생김.

Course와 Enrollment가 다른 서비스니까 객체 DI로 연결할 수 없다.

그래서 Enrollment → REST → Course 라는 통신이 필요해지고,

그 결과 GET /api/courses/internal/exists/{id}

같은 내부 Endpoint가 추가됨

Enrollment가 Course에 REST로 강의 존재 여부를 확인하도록 설계

"이 기능을 완성하려면 다른 서비스의 결과가 필요하네?" → **REST API** 필요

3. Sprint 1이 끝나면 실제로 한번 써본다(Walking Skeleton)

회원가입 → 강의목록 조회 → 강의 선택 → 수강신청 → PENDING 저장

결과까지 실제로 돌아가는거 확인

4. Sprint Review에서 부족한 점이 보인다

직접 돌려보니 "수강신청했는데 계속 PENDING이네?"

그러면 결제하면 수강 상태를 ACTIVE로 바꾸자.

Backlog 우선순위가 올라감.

5. Sprint 2 — 결제를 붙인다

이제 **Payment Service**를 추가.

필요한 기능:

수강신청 → 결제 요청 → 결제 성공 → 수강 활성화

여기서 다시 통신을 분석.

Enrollment가: "결제 요청하고 지금 결과를 알고 싶다."

그러면 즉시 응답이 필요하니까:

Enrollment → REST → **Payment**

자료에서도: **POST /api/payments/internal/request**

라는 내부 결제 요청 **Endpoint**를 가지고 있음

그런데 또 개발하다 보니까

Payment가 결제 완료됐다는 걸 **Enrollment**에게 알려줘야 함.

Payment → REST → **Enrollment**

"결제 끝났으니까 **ACTIVE**로 바꿔"

그런데 설계해보니: 결제 완료라는 사건을 **Enrollment**뿐 아니라 다른 서비스도 나중에 알아야 할 수도 있겠네?

그러면 **Kafka**가 더 적절하다고 판단

Payment → **payment.completed** → **Kafka** → **Enrollment** → **ACTIVE**

자료에서도 실제로 **Payment**가 **payment.completed**를 발행하고 **Enrollment**가 받아 **ACTIVE**로 변경하도록 되어 있어.

즉 기능 요구 → 통신 방식 결정

6. 그다음 추천을 추가

서비스가 어느 정도 돌아가니 이런 **User Story**가 올라옴 "수강생으로서 내가 좋아할 만한 강의를 추천받고 싶다."

그러면:

Recommend Service를 추가하고 Endpoint는 GET /recommend/{userId}

로 만들 수 있다.

추천 서비스는 자기 혼자서는 추천을 못 함

수강 이력 필요 → Enrollment 호출

강의 정보 필요 → Course 호출

그래서:

Recommend

└─ REST → Enrollment

└─ REST → Course

구조

실제 자료의 추천 흐름도 수강 이력을 Enrollment에서 조회하고 Course에서 강의 정보를 조회해서 추천 결과를 만드는 방식.

7. 전체 흐름 정리

User Story "강의를 조회하고 싶다" → 기능 분석 "강의 목록 조회" → 담당 서비스 Course Service → REST Endpoint, GET /api/courses → Controller @GetMapping → Service → Repository → DB

User Story "수강신청하고 싶다" → Enrollment Service 필요 → POST /api/enrollments → 개발하다 보니 강의 확인 필요 → Course Service와 통신 필요 → REST → GET /api/courses/internal/exists/{id}

점진적으로 시스템이 커짐.

Chapter 5. 궁금한점 및 보충

지금 코드는 다른 환경변수간 데이터베이스 공유는 안되는 형식이 맞나?

현재처럼 각 환경에서 **MariaDB** 컨테이너를 따로 실행하면 환경별 **DB** 데이터는 서로 공유되지 않는다.

DB 이름·아이디·비밀번호가 같아도 **DB** 서버가 다르면 데이터는 별개다.

여러 환경에서 같은 데이터를 쓰려면 동일한 원격 **DB** 서버를 바라보도록 설정해야 한다

도커가 굉장히 많다 이거 서비스별로 포트번호 각각 다 다르게 해야되고 이전에 같은 포트로 돌아가고 있는게 있으면 종료해야하나?

MSA에서는 서비스마다 컨테이너가 따로 존재한다.

컨테이너 내부 포트는 서로 같아도 가능하지만, 같은 **PC**에 공개하는 **Host Port**는 중복될 수 없다.

예를 들어 이미 **8080**이나 **3379**를 사용하는 컨테이너가 있다면 종료하거나 다른 **Host Port**로 변경해야 한다.

지금 뜬거는 서비스별로 컨테이너 라는거 알겠는데 **eurek** 이랑 **kafka** 라는게 새로보인다 무슨 역할이나?

Docker Compose: 여러 컨테이너의 실행, 네트워크, 포트, 환경변수 등을 한 번에 구성하고 연결한다.

Eureka: 서비스 등록소. 각 서비스가 어디에서 실행 중인지 등록하고 다른 서비스가 위치를 찾을 수 있게 한다.

Kafka: 서비스 사이에서 이벤트를 전달하는 메시징 시스템이다.

서비스 호출은 **A**가 **B**를 직접 찾아가서 “이 작업을 해줘”라고 요청하는 방식이고, 이벤트 통신은 **A**가 **Kafka**에 “이 일이 발생했다”고 알리면 그 이벤트를 구독한 **B·C** 등이 각자 필요한 작업을 수행하는 방식

어제 민감한 정보를 숨기는 **.env**파일로 도커를 실행해봤다. 근데 배포를 할때 다른 환경에서도 돌아갈려면 결국 거기서도 **.env**알아야 되지않나?**.env**는 보통 본인환경 께로 설정하는 것들인가?

.env는 DB 비밀번호, API Key 등 설정값과 민감정보를 코드에서 분리하기 위한 파일이다.

배포 환경에서도 필요한 환경변수 값은 있어야 하지만, 개발자의 **.env** 파일을 그대로 전달하는 것은 아니다.

개발·테스트·운영 환경이 각각 자기 환경에 맞는 값을 따로 설정하는 것이 일반적이다.

즉 **.env** 형식은 공유할 수 있지만 실제 비밀값은 환경마다 별도로 관리한다.

rest api

REST API는 클라이언트와 서버를 연결하는 기술이라기보다, HTTP 기반으로 "요청하는 프로그램"과 "응답하는 프로그램"을 연결하는 방식이다. 따라서 MSA에서는 한 서버가 다른 서버의 클라이언트가 되어 컨테이너 간에도 REST API를 사용할 수 있다.